

Universidade do Minho
Escola de Engenharia

Processamento de Linguagens

Trabalho Prático - Grupo 16

David Lopes Machado	(A107325)
João Pedro Araújo Fernandes	(A103568)
Rodrigo de Sousa Campos Pacheco da Rocha	(A107335)

17 de maio de 2026

Conteúdo

1	Introdução	2
2	Arquitetura do Compilador	2
2.1	Gestão centralizada de erros	2
3	Análise Léxica	3
3.1	Formato Suportado	3
3.2	Tokens reconhecidos	3
3.3	Caso Especial: Labels	3
3.4	Tratamento de Erros Léxicos	3
4	Análise Sintática	4
4.1	Estrutura da Gramática	4
4.2	Construção da AST	4
4.3	Precedência de Operadores	5
4.4	Resolução de Ambiguidades: Arrays vs Funções	5
4.5	Tratamento de Erros Sintáticos	5
5	Análise Semântica	5
5.1	Symbol Table	5
5.2	Padrão de Visita	6
5.3	Verificação de tipos	6
5.4	Resolução da Ambiguidade Arrays vs Funções	6
5.5	Validação das Labels	6
5.6	Tratamento de Erros e Avisos	7
6	Otimização	7
6.1	Eliminação de Código Morto	7
6.2	Remoção de Declarações não Utilizadas	7
6.3	Constant Folding	7
7	Geração de Código	8
7.1	Gestão de Memória e Acesso a Dados	8
7.2	Implementação de Estruturas de Controlo	8
7.3	Ciclos e Protocolo de Chamada de Subprogramas	8
8	Guia de Execução	9
9	Conclusão	9

1 Introdução

O presente relatório aborda o trabalho prático da cadeira de Processamento de Linguagens, cujo objetivo passa por desenvolver um compilador de Fortran 77 Standard em Python, utilizando a ferramenta PLY (Python Lex-Yacc). Neste documento está especificada a arquitetura do compilador desenvolvido, composta pelas fases de análise léxica, sintática, semântica, otimização e geração de código e documentadas as decisões de implementação tomadas em cada fase. Adicionalmente, no final analisamos as dificuldades encontradas durante o desenvolvimento e explicamos como correr o nosso compilador.

2 Arquitetura do Compilador

A arquitetura do compilador divide-se em cinco fases principais sequenciais: análise léxica, análise sintática, análise semântica, otimização e geração de código. Transversalmente a estas camadas, o processo é suportado por duas estruturas fundamentais: a Symbol Table, que mantém o registo e estado de todas as entidades (variáveis, arrays, subprogramas), e um módulo centralizado de erros. A arquitetura global do sistema encontra-se ilustrada no diagrama abaixo:

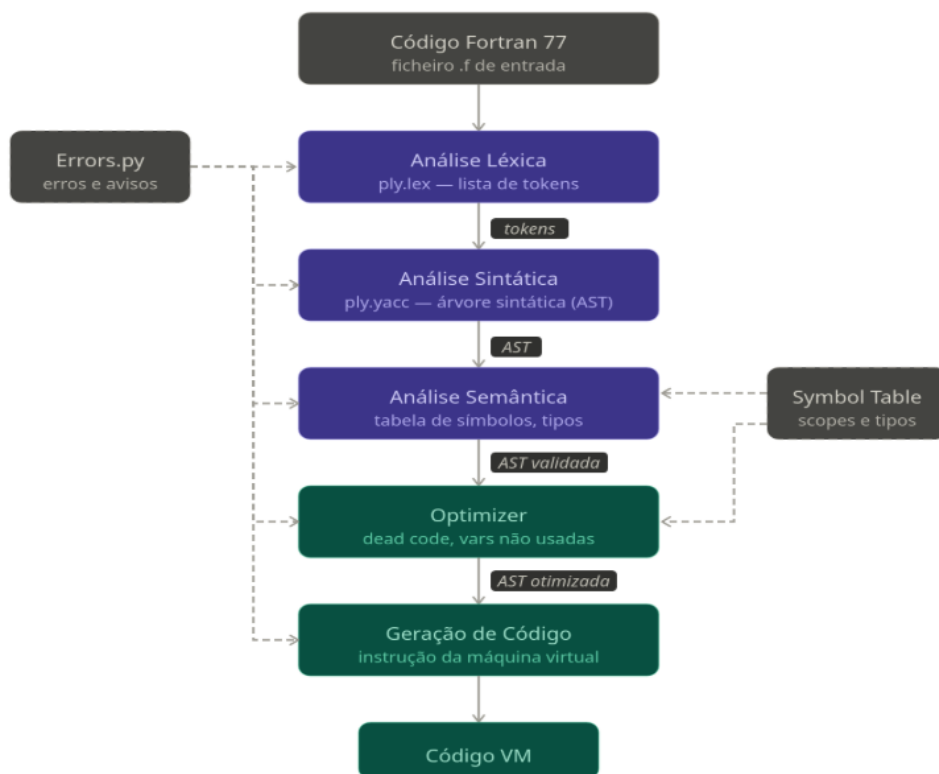


Figura 1: Arquitetura do Compilador

2.1 Gestão centralizada de erros

Para garantir consistência e resiliência ao longo de todas as fases, a gestão de erros e avisos foi centralizada no módulo Errors.py. O compilador adota uma estratégia de recuperação (panic mode / skip), o que permite que a compilação não seja interrompida ao primeiro erro léxico ou semântico encontrado. Em vez disso, o compilador regista os problemas (como caracteres inválidos ou variáveis não declaradas) e continua a análise, maximizando o feedback devolvido ao utilizador numa única execução. Erros sintáticos críticos (como blocos mal fechados) lançam exceções diretas para garantir que não é gerada uma Árvore de Sintaxe Abstrata (AST) corrompida.

3 Análise Léxica

A análise léxica é a primeira fase do compilador, sendo responsável por transformar o código Fortran numa sequência de tokens que o parser possa ler. Para implementar esta camada foi utilizado, tal como requisitado, a biblioteca PLY do Python.

3.1 Formato Suportado

Relativamente ao formato da linguagem, o grupo optou por suportar o formato free-form das versões mais modernas, não implementando as restrições posicionais do fixed-form. Esta decisão foi tomada para simplificar significativamente a implementação. Consequentemente, para os comentários é suportado o carácter “!”, seguindo a convenção do free-form.

3.2 Tokens reconhecidos

O analisador léxico foi configurado para reconhecer os elementos fundamentais da linguagem através do agrupamento de expressões regulares. Para implementar esta parte do projeto, os tokens foram divididos nas seguintes categorias:

Literais e Delimitadores: Caracteres únicos capturados de forma direta, essenciais para a estrutura sintática, tais como operadores aritméticos básicos (+, -, *, /), parênteses e pontuação matemática (=, ,, :).

Operadores Relacionais e Lógicos: Identificados pelas expressões padrão do Fortran delimitadas por pontos (ex: .EQ., .AND., .NOT.). Foi garantida a sua definição prioritária no código face aos identificadores, evitando ambiguidades na captura léxica. Foi também definido o operador de potência (**), essencial para expressões aritméticas.

Tipos de Dados e Identificadores: Regras para a extração de valores inteiros (INTVAL), reais (REALVAL), booleanos (BOOLEAN), cadeias de caracteres (STRING) e variáveis (VAR).

Palavras Reservadas: Para evitar a definição de dezenas de expressões regulares individuais, optou-se por capturar todas as palavras reservadas (como PROGRAM, IF, DO, INTEGER, etc.) através da regra genérica das variáveis (t_VAR). O lexer verifica se o identificador capturado existe num dicionário pré-definido de palavras reservadas; caso exista, o tipo do token é atualizado adequadamente, sendo caso contrário tratado como uma variável comum.

3.3 Caso Especial: Labels

Um dos desafios encontrados na implementação léxica foi a distinção entre um label e um número inteiro no meio da expressão. Em Fortran 77, os labels são números que aparecem no início de uma linha antes de uma instrução, como em “10 CONTINUE” ou “20 IF (...)”. No entanto, o lexer precisa de distinguir um inteiro numa label de um inteiro, por exemplo, numa conta de multiplicação. A solução implementada consistiu em avaliar o contexto do token dentro da regra de reconhecimento dos inteiros (t_INTVAL). Através do atributo line_start, que guarda a posição do início da linha atual, o lexer extrai o texto que antecede o número. Se esse texto for vazio ou contiver apenas espaços em branco (garantindo que o número é o primeiro elemento útil da linha), e o carácter imediatamente a seguir for um espaço ou tabulação, o token é reclassificado de INTVAL para LABEL.

3.4 Tratamento de Erros Léxicos

Para garantir a resiliência do analisador, foi implementada a função t_error, responsável por capturar caracteres não reconhecidos. Quando um símbolo ilegal é detetado, o incidente é reportado através do módulo auxiliar Errors, indicando a linha e o carácter inválido no terminal. De forma a maximizar o feedback numa única execução, o lexer incrementa um contador interno e

descarta o caráter problemático (recorrendo ao método skip), permitindo que a análise prossiga sem interromper a compilação e identifique eventuais falhas subsequentes no código.

4 Análise Sintática

A análise sintática foi implementada com recurso ao módulo yacc da biblioteca PLY. Este módulo recebe a sequência de tokens gerados pelo lexer e verifica se estes formam expressões válidas de acordo com a gramática definida para o nosso compilador, construindo simultaneamente a Árvore de Sintaxe Abstrata (AST).

4.1 Estrutura da Gramática

A gramática do compilador foi desenhada na ferramenta ply.yacc, que implementa um analisador sintático (parser) de abordagem bottom-up, com o objetivo de espelhar rigorosamente a arquitetura estipulada pelo standard do Fortran 77. A regra de partida (file) define que um ficheiro de código-fonte é composto por uma sequência de uma ou mais unidades de programa (program_unit), que podem assumir a forma de um programa principal (PROGRAM), de uma função (FUNCTION) ou de uma subrotina (SUBROUTINE). Em estrita concordância com o standard, estas unidades são processadas linearmente ao mesmo nível gramatical, garantindo estruturalmente a inexistência de aninhamento de subprogramas.

Uma das características mais rígidas do Fortran 77 é a divisão interna obrigatória de cada unidade de programa. A gramática desenvolvida reflete esta exigência forçando a existência de uma secção de declarações (declaration_section) que tem de preceder estritamente a secção executável (executable_section). Esta modelação garante, logo na fase de análise sintática, que qualquer instrução executável colocada indevidamente no meio do bloco de declarações desencadeia um erro imediato. Para processar estas secções e outras sequências de elementos com tamanho indefinido, como listas de parâmetros ou blocos de instruções, a gramática tira partido de regras com recursividade à esquerda (por exemplo: executable_section : executable_section statement | empty). Esta abordagem foi escolhida de forma deliberada exatamente devido à natureza bottom-up do analisador sintático construído pelo PLY, cenário onde a recursividade à esquerda é processada de forma iterativa e muito mais eficiente, consumindo significativamente menos memória na stack do que a recursividade à direita.

Ao nível das declarações, a gramática foi desenhada para ser compacta e expressiva. A regra var_decl consegue capturar simultaneamente variáveis escalares tradicionais e declarações de arrays unidimensionais estáticos, o que permite que a AST gerada classifique imediatamente a entidade com a tag adequada ('SCALAR' ou 'ARRAY'). Adicionalmente, como a linguagem Fortran 77 depende fortemente de labels numéricos para o controlo de fluxo, a gramática abstrai as instruções na regra statement, que bifurca em instruções com ou sem identificador numérico (labeled_statement ou unlabeled_statement). Desta forma, qualquer instrução válida da linguagem pode, de forma opcional e uniforme, ser antecedida por um label, permitindo que o nó resultante na AST capture esse número e embrulhe a instrução interna, mantendo a árvore sintática coerente, linear e perfeitamente estruturada para a posterior fase de otimização.

4.2 Construção da AST

Neste módulo optou-se por criar uma Árvore Sintática Abstrata (AST), em oposição à execução da análise semântica imediatamente durante o processo de parsing, para respeitar a modularidade. Avaliar as regras semânticas, enquanto a gramática é resolvida, misturaria a lógica de validação de tipos com a estrutura sintática, resultando num código complexo e inflexível.

A AST foi então implementada através de tuplos nativos em Python assegurando imutabilidade e eficiência na travessia. Cada nó segue uma estrutura definida, cujo primeiro elemento é um identificador (ex: 'IF', 'ASSIGN'), crucial para o processamento nas fases seguintes. A

árvore descarta elementos não necessários, como os parênteses ou as palavras THEN e ENDIF. Um bloco condicional, por exemplo, é simplificado para: ('IF', condição, bloco_then, bloco_else, linha). O registo da linha em alguns nós é determinante para o registo de erros na fase seguinte.

4.3 Precedência de Operadores

A tabela de precedências foi definida explicitamente no PLY para resolver ambiguidades em expressões compostas. Os operadores lógicos (OR, AND, NOT) têm menor precedência, seguidos dos relacionais (.LT., .EQ., etc.), depois os aritméticos (+, -, *, /, **). O operador unário de negação (UMINUS) tem a maior precedência aritmética, sendo associativo à direita. Esta hierarquia segue o comportamento esperado do Fortran 77.

4.4 Resolução de Ambiguidades: Arrays vs Funções

Um problema detetado nesta fase foi a ambiguidade entre arrays e funções na sua chamada. Tanto um array NUMS(I) como uma função FUN(X) têm a mesma sintaxe no parser. De forma a resolver esta ambiguidade tomou-se a decisão de passar a responsabilidade para o analisador semântico, consultado a tabela de símbolos para distinguir os dois, o que será explicado na respetiva secção.

4.5 Tratamento de Erros Sintáticos

Relativamente aos erros sintáticos, optou-se por lançar exceções com mensagens claras em vez de tentar recuperar e continuar a análise. Foram definidas regras de erro específicas para as situações mais comuns: END em falta num bloco PROGRAM ou FUNCTION, THEN em falta após a condição do IF, e ENDIF em falta para fechar o bloco condicional. Para todos os restantes casos, a função p_error genérica reporta o token inesperado e a linha onde ocorreu, ou sinaliza um fim de ficheiro inesperado caso o código termine de forma incompleta.

5 Análise Semântica

A análise semântica é a terceira fase do compilador, responsável por verificar a coerência semântica do programa. Esta fase, recebendo a AST gerada pelo parser percorre-a e valida o que as regras de gramática não conseguem por si só, como as declarações de variáveis, a compatibilidade de tipos em expressões e atribuições, ou a validade dos labels. Para implementar esta análise, dois componentes essenciais foram necessários: a Symbol Table, responsável por registar e organizar toda a informação relativa aos identificadores do programa, e o analisador semântico, encarregue de percorrer a AST e efetivamente validá-la.

5.1 Symbol Table

A Symbol Table foi implementada com uma estrutura de scopes, onde existe sempre um scope global para o programa geral e um scope adicional para cada função e subrotina. No momento em que o analisador entra numa função ou subrotina o scope é atualizado para o da mesma, voltando ao global quando a função termina. Cada entrada na tabela regista uma variável ou subprograma, mantendo o tipo, se é um array e o seu tamanho, a linha onde foi declarada e um campo “used” que indica se foi utilizada. Este último campo é posteriormente utilizado para implementar otimizações.

Foram encontrados dois problemas relacionados com a variável de retorno de funções. O primeiro consistia em que, dentro de uma função FUN(X), a instrução de retorno FUN = valor levava o analisador a procurar a variável FUN na symbol table. Como ela não existia no scope local da função, o mecanismo de lookup subia ao scope global onde encontrava a entrada da

própria função, marcando-a incorretamente como utilizada. Isto impedia o otimizador de remover funções nunca chamadas. O segundo problema era que, no final da análise, o método `check unused variables` detetava esta variável de retorno como nunca atribuída e gerava um erro semântico indevido. A solução para ambos foi injetar automaticamente uma variável com o nome da função no seu próprio scope local aquando da sua declaração, representando a variável de retorno. Desta forma, o `FUN = valor` resolve-se localmente sem subir ao global, e o campo “used” da função no scope global só é ativado quando existe uma chamada externa real.

5.2 Padrão de Visita

Para percorrer a AST, o analisador semântico implementa o padrão de visita através de um método central (`visit`). Este método recebe um nodo da AST, extrai o seu identificador e constrói o nome do método que deve invocar (o nodo ‘IF’ é direcionado para o método `visit_IF`). Caso não exista um método específico para o identificador, o método central retorna ‘UNKNOWN’ e a análise continua. O tipo especial ‘UNKNOWN’ é também retornado sempre que é detetado um erro em qualquer nodo, de forma a poder registar o erro e não interromper a análise, permitindo que seja detetado o máximo de problemas possíveis. Esta abordagem mantém o código extensível e modular, proporcionando uma análise de erros detalhada ao utilizador.

5.3 Verificação de tipos

A verificação de tipos é aplicada em todas as expressões e atribuições do programa. Nas operações aritméticas (+, -, *, /, **), ambos os operandos devem ser numéricos, devolvendo `INTEGER` se ambos forem inteiros, ou `REAL` caso contrário. Nas operações relacionais (.LT., .EQ., etc.), os operandos devem ser numéricos e o resultado é sempre `LOGICAL`. Nas operações lógicas (AND, OR, NOT), os operandos devem ser do tipo `LOGICAL`. Nas atribuições, o tipo da expressão deve ser compatível com o tipo da variável destino. A única exceção permitida é a mistura entre `INTEGER` e `REAL`, sendo gerado um aviso quando se atribui um valor `REAL` a uma variável `INTEGER`, alertando o utilizador para a truncagem do valor.

5.4 Resolução da Ambiguidade Arrays vs Funções

Como foi mencionado na secção 4.4, a sintaxe de acesso a um array e de chamada de uma função é gramaticalmente idêntica, sendo ambas representadas pelo mesmo tipo de nodo (`CALL`). A distinção é feita no método `visit_CALL`, onde a symbol table é consultada para determinar a natureza do identificador. Se a entrada tiver o campo “is_array” ativo, o nodo é tratado como um acesso a um array, sendo validado o tipo do índice e os limites do array. Caso contrário, o nodo é tratado como uma chamada a uma função, sendo validados os argumentos e devolvido o tipo de retorno registado. Para suportar chamadas a funções declaradas após o ponto de chamada, padrão comum em Fortran 77, o analisador adia a validação dessas chamadas. Durante a travessia, as chamadas a identificadores desconhecidos são guardadas numa lista de chamadas pendentes. No final da análise, o método `check_pending_calls` verifica se cada função foi entretanto declarada e se o número de argumentos é correto. De forma análoga, as chamadas a subrotinas através da instrução `CALL` são validadas pelo método `visit_CALL_STMT`, que verifica se a subrotina foi declarada e se o número de argumentos fornecidos é correto, seguindo a mesma lógica de chamadas pendentes para subrotinas declaradas após o ponto de chamada.

5.5 Validação das Labels

Outra das dificuldades desta fase foi a validação dos labels, uma vez que a sua declaração e utilização podem estar separadas por várias linhas de código, impossibilitando a sua validação no momento em que são encontrados. Portanto, durante a travessia da AST, o analisador regista em conjuntos diferentes os labels realmente declarados, os labels esperados pelos ciclos `DO`, e os

labels esperados pelas instruções GOTO. No final da análise, o método `check_unresolved_labels` verifica os conjuntos e reporta qualquer label que foi referenciado e nunca declarado.

5.6 Tratamento de Erros e Avisos

Como referido na secção 5.2, o analisador nunca interrompe a análise quando deteta um erro, em vez disso, regista-o numa lista interna e retorna um tipo especial, continuando a travessia e detetando o máximo de erros possíveis. O mesmo princípio se aplica aos avisos, que possuem uma lista separada. Contudo, enquanto que os erros impedem a geração de código, os avisos são meramente informativos. Os avisos implementados incluem a truncagem silenciosa de um REAL para INTEGER numa atribuição, e a declaração de uma variável ou subprograma não utilizada.

6 Otimização

Após acabar a análise semântica e validar a AST, o código passa para a fase de otimização, produzindo uma nova AST otimizada. O otimizador reutiliza a symbol table já preenchida, evitando repetições. Esta fase é extremamente importante pois permite reduzir o tamanho e a complexidade do código gerado, eliminando instruções e declarações desnecessárias antes de chegar ao gerador de código.

6.1 Eliminação de Código Morto

Em Fortran 77 um GOTO torna o código que lhe sucede inalcançável até ao próximo LABEL. Logo, o otimizador deteta qualquer GOTO que apareça diretamente numa lista de statements e remove tudo o que lhe sucede até ao próximo LABEL. Esta lógica é aplicada recursivamente dentro de blocos IF e LABEL, garantindo que o código morto é sempre eliminado. Adicionalmente, quando a condição de um bloco IF é uma constante booleana conhecida em tempo de compilação, o otimizador elimina o ramo que nunca será executado, no caso de IF (.TRUE.) o ramo ELSE é removido, e no caso de IF (.FALSE.) o ramo THEN é removido, podendo o bloco IF inteiro desaparecer caso não exista ELSE.

6.2 Remoção de Declarações não Utilizadas

Para cada variável ou subprograma, o otimizador acede à symbol table preenchida e verifica o respetivo campo “used”. Se o campo estiver a False, a entidade é removida da AST e da symbol table, mantendo-as sempre sincronizadas. No caso de declarações de variáveis, se apenas algumas variáveis de uma linha de declaração forem removidas, a declaração é mantida apenas com as variáveis utilizadas. Mas, se todas forem removidas, a declaração inteira desaparece da AST. As funções e subrotinas não utilizadas são removidas diretamente como unidades completas da AST.

6.3 Constant Folding

O constant folding consiste em avaliar em tempo de compilação expressões cujo valor é inteiramente conhecido, substituindo-as pelo resultado. O otimizador aplica esta técnica a expressões binárias onde ambos os operandos são constantes literais, cobrindo operações aritméticas (+, -, *, /, **), relacionais (.LT., .EQ., etc.) e lógicas (.AND., .OR.), bem como operações unárias de negação (UMINUS) e inversão lógica (.NOT.). A avaliação é feita recursivamente, pelo que expressões como $2 * (4 + 1)$ são primeiro simplificadas internamente ($4 + 1 \rightarrow 5$) e depois no nível superior ($2 * 5 \rightarrow 10$). Um caso especial é a divisão inteira, pois em Fortran 77, a divisão entre dois inteiros é inteira, por isso $7 / 2$ resulta em 3 e não 3.5, o que o otimizador respeita usando a divisão inteira do Python. Em caso de divisão por zero em tempo de compilação, o

otimizador aborda a situação de forma defensiva, registrando um aviso e deixando a expressão original na AST para que o erro ocorra em runtime. Esta otimização combina ainda com a eliminação de blocos IF — quando a condição se torna uma constante booleana após o folding, o ramo correspondente é eliminado automaticamente.

7 Geração de Código

A fase final do processo de compilação consiste na geração de código, cuja responsabilidade é traduzir a Árvore de Sintaxe Abstrata (AST), previamente validada e otimizada, numa sequência linear de instruções executáveis pela máquina virtual (VM) alvo. Para efetuar esta tradução, foi implementado um gerador que percorre a AST aplicando o padrão de projeto Visitor, uma abordagem análoga à utilizada nas fases de análise semântica e otimização. Este método permite que cada nodo da AST seja sistematicamente mapeado para uma ou mais instruções da VM, que operam sobre uma arquitetura de baixo nível baseada em stack.

7.1 Gestão de Memória e Acesso a Dados

A gestão da memória constitui um aspeto central desta fase, sendo a Symbol Table um recurso indispensável para determinar o endereço e o tipo de cada identificador. As variáveis declaradas no âmbito do PROGRAM principal são tratadas como globais, sendo o seu acesso mediado pelas instruções pushg e storeg, que operam sobre offsets num segmento de memória global. Em contrapartida, os parâmetros e as variáveis declaradas no interior de subprogramas (FUNCTION ou SUBROUTINE) são alocados numa stack frame local a cada chamada. O seu manuseamento é realizado através das instruções pushl e storel, que utilizam offsets relativos ao ponteiro da frame atual. O acesso a elementos de um array representa um caso mais complexo. Os arrays globais são alocados dinamicamente no heap através da instrução allocn, sendo o seu endereço base recuperado via pushst. As operações de leitura e escrita são então efetuadas pelas instruções loadn e storen, que operam sobre endereços heap e tratam os valores como inteiros, evitando conflitos de tipagem com a VM. Os arrays locais a subprogramas seguem uma abordagem análoga.

7.2 Implementação de Estruturas de Controlo

A tradução das estruturas de controlo de fluxo do Fortran para a linguagem da VM é alcançada através da utilização de saltos condicionais e incondicionais, coordenados por labels geradas dinamicamente pelo compilador. Uma estrutura IF-THEN-ELSE, por exemplo, é materializada pela geração de código que avalia a expressão condicional, cujo resultado booleano (1 para verdadeiro, 0 para falso) é deixado no topo da stack. Subsequentemente, uma instrução de salto condicional, como jz, para desviar o fluxo. O bloco THEN é então gerado, seguido por um salto incondicional (jump) que transpõe o bloco ELSE. Desta forma, garante-se a execução mutuamente exclusiva de apenas um dos ramos da estrutura condicional.

7.3 Ciclos e Protocolo de Chamada de Subprogramas

De forma similar, os ciclos DO são implementados através de um par de labels que delimitam o seu corpo, onde a condição de terminação é verificada no início de cada iteração e a variável de controlo é incrementada no final. O protocolo de chamada a subprogramas é igualmente rigoroso, garantindo a correta passagem de argumentos e gestão do fluxo de controlo. Antes da chamada (call), os argumentos são avaliados e os seus valores colocados na stack. No prólogo da função chamada, é alocado espaço para as suas variáveis locais (pushn). Os parâmetros recebidos são acedidos através de offsets negativos relativos ao ponteiro de frame (fp), uma vez que foram empurrados para a stack antes da instrução call, enquanto as variáveis locais ocupam offsets positivos alocados pelo pushn. A instrução return encerra a execução do subprograma, desalocando

a sua stack frame, devolvendo o controlo ao chamador e, no caso de uma FUNCTION, deixando o seu valor de retorno no topo da stack principal. Este mecanismo assegura o isolamento de cada invocação e suporta naturalmente a recursividade.

8 Guia de Execução

O compilador é executado através do script `main.py`, localizado na pasta `src/`. A forma de invocação é a seguinte:

```
python3 src/main.py <ficheiro_entrada> [opções]
```

Opções disponíveis:

- `-o, -output <ficheiro>` - Define o nome do ficheiro de saída para o código VM. Por omissão, é gerado um ficheiro com o mesmo nome do ficheiro de entrada e extensão `.vm`.
- `-no-opt` - Desativa todas as otimizações, gerando código diretamente a partir da AST semântica.
- `-no-warn` - Suprime todos os avisos, mostrando apenas erros.

Durante a execução, o compilador reporta o progresso de cada fase no terminal. Em caso de erro léxico, sintático ou semântico, a compilação é interrompida com uma mensagem descritiva. Se todas as fases forem concluídas com sucesso, o código VM é guardado no ficheiro de saída e uma mensagem de confirmação é apresentada.

9 Conclusão

O desenvolvimento deste compilador para Fortran 77 permitiu ao grupo aplicar na prática os conceitos estudados ao longo da unidade curricular. O resultado é um compilador funcional que cobre todas as etapas obrigatórias, análise léxica, sintática, semântica e geração de código, bem como a fase de otimização como valorização, incorporando eliminação de código morto, remoção de declarações não utilizadas e constant folding. Ao longo do desenvolvimento, os principais desafios encontrados foram a ambiguidade entre arrays e funções, a validação diferida de labels e chamadas a subprogramas, e a gestão correta da variável de retorno de funções. A correção do compilador foi validada através de um conjunto alargado de testes automatizados, cobrindo os exemplos do enunciado e diversas situações de erro, tendo todos sido concluídos com sucesso.